QUESTION 1

Which of the following data types would be most memory-efficient for storing the small integer number in an array, given that the maximum value of numbers can be 50?

Not Attempted **Score: 0 / 1**

Options

A. int

B. unsigned int

Expected

C. uint8_t

D. uint16_t

FEEDBACK

Explanation: uint8_t can store values from 0 to 255, which is sufficient for counting up to 50 items and uses only 1 byte of memory, making it more memory-efficient than int or uint16_t.

QUESTION 2

Embedded systems often prioritize efficient code execution. How do loops generally compare to function calls in terms of execution speed?

Not Attempted **Score: 0 / 1**

Options

A. Loops are always faster than function calls due to simpler execution flow.

Expected

B. Function calls introduce some overhead for argument passing and function setup.

C. Loops are slower than function calls, as they require repeated evaluation of conditions.

D. There's no inherent difference in speed between loops and function calls.

FEEDBACK

Explanation: Loops directly execute the code within the loop body. Function calls involve additional steps like argument passing, function prologue/epilogue, and potential stack manipulation. This overhead can introduce a slight performance penalty.

QUESTION 3

Consider an embedded system that needs to store configuration data that never changes during operation. Which memory type would be most suitable?

Not Attempted **Score: 0 / 1**

Options

A. Random Access Memory (RAM).

Expected

B. Read-Only Memory (ROM).

C. Flash memory.

D. Dynamic Random Access Memory (DRAM).

FEEDBACK

Explanation: ROM is non-volatile memory, meaning it retains data even after power loss. This makes it ideal for storing configuration data that rarely changes. Flash memory offers a balance between re-writability and retention, but for strictly static data, ROM is a more efficient choice.

QUESTION 4

What is the primary difference between a struct and a union in C?

Not Attempted **Score: 0 / 1**

Options

Expected

A. struct allocates memory for each member separately, while union shares the same memory space for all its members.

B. struct can only hold integer types, while union can hold any data type.

C. struct is used for hardware interfacing, while union is used for software configuration.

D. struct members are accessed sequentially, while union members are accessed randomly

FEEDBACK

Explanation: In a struct, each member has its own memory allocation, and the total size of the struct is the sum of the sizes of its members. In a union, all members share the same memory space, and the size of the union is determined by the size of its largest member. This allows efficient use of memory when members are not used simultaneously.

QUESTION 5

Which of the following correctly dereferences a void pointer after typecasting it to an integer pointer?

Not Attempted **Score: 0 / 1**

Options

A. *(void *)p

Expected

B. *(int *)p

C. *p

D. (int)p

FEEDBACK

Explanation: To dereference a void pointer after typecasting it to an integer pointer, use *(int *)p.

QUESTION 6

What is the output of the following code? Consider the little-endian memory representation.

```
int x = 0x12345678;
short *p = (short*)&x;
printf("%X", *p);
```

Not Attempted **Score: 0 / 1**

Options

A. 1234

Expected

B. 5678

C. Undefined behavior

D. 3456

FEEDBACK

Explanation: The integer x is typecast to a short pointer. Accessing *p retrieves the lower 16 bits of the integer x, which are 0x5678 in little-endian memory representation.

QUESTION 7

What is the result of 5 | 3?

Not Attempted **Score: 0 / 1**

Options

A. 0

B. 5

Expected

C. 7

D. 15

FEEDBACK

Explanation: The | operator performs a bitwise OR. The binary representation of 5 is 0101 and for 3 is 0011. Performing a bitwise OR gives 0111, which is 7 in decimal.

QUESTION 8

What is the result of 0xF0 >> 4?

Not Attempted **Score: 0 / 1**

Options

Expected

A. 0x0F

B. 0xFF

C. 0xF0

D. 0x1F

FEEDBACK

Explanation: The right shift operator >> shifts bits to the right. For 0xF0 (11110000 in binary), shifting 4 bits to the right results in 00001111, which is 0x0F in hexadecimal.

QUESTION 9

What should be the mask to set the bit number 4 from right(in 0-indexing) of a byte x to 1?

Not Attempted **Score: 0 / 1**

Options

A. 0x04

B. 0x08

C. 0x0C

Expected

D. 0x10

FEEDBACK

Explanation: To set the 5th bit (which is the 4th bit in 0-indexing), you can use the bitwise OR with a mask where only the 5th bit is 1 (0x10 which is 00010000 in binary).

QUESTION 10

Consider the following code snippet to check the integer n is even or odd.

```
if(n%2!=0)
      printf("odd");
else
      printf("even");
```

Which of the following code is equivalent to the given code for output?

Not Attempted **Score: 0 / 1**

Options

Expected

A.

```
if(n & 1)
      printf("odd");
 else
      printf("even");
```

B.

```
if(n | 1)
      printf("odd");
 else
      printf("even");
```

C.

```
if(n ^ 1)
      printf("odd");
 else
      printf("even");
```

FEEDBACK

Explanation: The bitwise AND operation n & 1 checks the least significant bit (LSB) of the integer n. If the LSB is 1, the number is odd; if the LSB is 0, the number is even. This is because in binary, odd numbers have their least significant bit set to 1, while even numbers have it set to 0.

Here is a detailed explanation of why the other options are incorrect:

B) if(n | 1): The bitwise OR operation n | 1 sets the least significant bit to 1 regardless of whether n is odd or even. Therefore, n | 1 is always non-zero (true), so it would always print "odd", which is incorrect.
C) if(n ^ 1): The bitwise XOR operation n ^ 1 toggles the least significant bit. This does not directly determine if n is even or odd, but instead gives a different number. The result of n ^ 1 being non-zero does not provide the correct condition for checking even or odd numbers.

Thus, the correct choice is (A)